Curs 3B

Programare Orientată pe Obiecte în limbajul Java

Programare Orientată pe Obiecte

Cuprins

- Transmiterea parametrilor
- Clase Wrapper
- Variabile si metode de clasa vs instanta
- Agregare si mostenire
- Polimorfism

Trasmiterea parametrilor (1)

TipReturnat metoda([Tip1 arg1, Tip2 arg2, ...])

- **Argumentele sunt trimise doar prin valoare (pass-by-value).**

```
void metoda(StringBuffer sir, int numar) {
    // StringBuffer este tip referinta
    // int este tip primitiv
    sir.append("abc");
    numar = 123;
}
...
StringBuffer s=new StringBuffer();
int n=0;
metoda(s, n);
System.out.println(s + ", " + n);
// s va fi "abc", dar n va fi 0
```

String / StringBuffer

String - Immutable

- Unui obiect de tip String nu i se poate modifica continutul! Se creaza un nou obiect String modificat!

StringBuffer – Mutable

- Unui obiect de tip StringBuffer i se poate modifica continutul!

Exemplu:

```
String s= new String ("test");
StringBuffer sb= new StringBuffer ("test");
s.concat("unu");
sb.append("unu");
System.out.println(s);  //test
System.out.println(sb); // testunu
System.out.println(s.concat("unu"));   // ?
```

Transmiterea parametrilor (2)

```
void metoda(String sir, int numar) {
    // String este tip referinta
    // int este tip primitiv
    sir = "abc";
    numar = 123;
}

...

String s=new String(); int n=0;
metoda(s, n);
System.out.println(s + ", " + n);

// s va fi "", n va fi 0
```

Transmiterea parametrilor (3)

```
void schimba(int a, int b) {
    int aux = a;
    a = b;
    b = aux;
}

...
int a=1, b=2;
schimba(a, b);
// a va ramane 1, iar b va ramane 2
```

Cum rezolvam in Java?

Transmiterea parametrilor (4)

```
class Pereche {
    public int a, b;
}
...

void schimba(Pereche p) {
    int aux = p.a;
    p.a = p.b;
    p.b = aux;
}
...

Pereche p = new Pereche();
p.a = 1, p.b = 2;
schimba(p);
//p.a va fi 2, p.b va fi 1
```

Metode cu număr variabil de argumente

[modif] TipReturnat metoda(TipArgumente ... args)

- Tipul argumentelor poate fi referinţă sau primitiv.

```
void metoda(Object ... args) {
    for(int i=0; i<args.length; i++)
        System.out.println(args[i]);
}

...

metoda("Hello");

metoda("Hello", "Java", 1.5);
```

Clase wrapper

- Set de clase folosite la ambalarea (wrap) valorilor primitive într-un obiect.

byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

- Toate clasele wrapper sunt imuabile (immutable), de fiecare data când modificati valoarea unui obiect wrapper este creat un obiect nou

Obs: Vector nu Vector<Integer> etc

Exemplu immutable

```
Integer intObject1 = 2300;     //autoboxing
Integer intObject2 = intObject1;
if ( intObject1 == intObject2 )  // ce compara?
    System.out.println("Acelasi obiect");
 else System.out.println("Obiecte diferite");

intObject1++;
  //deoarece este immutable se obtine un nou obiect

System.out.println(intObject1);
System.out.println(intObject2);
if ( intObject1 == intObject2 )
    System.out.println("Acelasi obiect");
 else System.out.println("Obiecte diferite");
```

Rezultat

Output:

Acelasi obiect

2301

2300

Obiecte diferite

Explicaţie:

- Compilatorul despacheteaza obiectul într-un primitiv
- Ii modifică valoarea
- Impachetează noua valoare într-un obiect nou
- Referinţa va primi adresa obiectului nou creat

Variabile de instanţă şi variabile de clasă

```
class Exemplu {
    int x ; //variabila de instanta
}
```

- variabilă de instanţă: la fiecare creare a unui obiect al clasei Exemplu sistemul alocă o zonă de memorie separată pentru memorarea valorii lui x.

```
class Exemplu {
    static int sx ; //variabila de clasă
}
```

- Pentru variabilele de clasă (statice) sistemul alocă o singură zonă de memorie la care au acces toate instanţele clasei respective, ceea ce înseamnă că dacă un obiect modifică valoarea unei variabile statice ea se va modifica şi pentru toate celelalte obiecte.

Variabile de clasă

- Deoarece nu depind de o anumită instanţă a unei clase, variabilele statice pot fi referite şi sub forma:

  NumeClasa.numeVariabilaStatica

  Ex. Exemplu.sx

- Iniţializarea variabilelor de clasă se face o singură dată, la încărcarea în memorie a clasei respective

```
class Exemplu {
    static final double PI = 3.14;
    static long nrInstante = 0;
    static Point p = new Point(0,0);
}
```

Variabile de instanţă şi variabile de clasă

```
class Exemplu {
    int x ; // Variabila de instanta
    static long n; // Variabila de clasa
}

    ...
    Exemplu o1 = new Exemplu();
    Exemplu o2 = new Exemplu();
    o1.x = 100;
    o2.x = 200;
    System.out.println(o1.x); // Afiseaza 100
    System.out.println(o2.x); // Afiseaza 200
    o1.n = 100;
    System.out.println(o2.n); // Afiseaza 100
    o2.n = 200;
    System.out.println(o1.n); // Afiseaza 200
    System.out.println(Exemplu.n); // Afiseaza 200
    // o1.n, o2.n si Exemplu.n sunt referinte la aceeasi
        // valoare
```

Metode de instanţă şi metode de clasă

- **metodele de instanţă operează atât pe variabilele de instanţă cât şi pe cele statice ale clasei;**
- **metodele de clasă operează doar pe variabilele statice ale clasei.**

```
class Exemplu {
    int x ;      // Variabilă de instanţă
    static long n; // Variabilă de clasă
    void metodaDeInstanta() {
        n ++; // Corect
        x --; // Corect
    }
    static void metodaStatica() {
        n ++; // Corect
        x --; // Eroare la compilare !
    }
}
```

Metode de instanţă şi metode de clasă

- Intocmai ca şi la variabilele statice, întrucât metodele de clasă nu depind de starea obiectelor clasei respective, apelul lor se poate face şi sub forma:

  NumeClasa.numeMetodaStatica

```
Exemplu.metodaStatica(); // Corect
Exemplu obj = new Exemplu();
obj.metodaStatica(); // Corect
```

- Metodele de instanţă nu pot fi apelate decât pentru un obiect al clasei respective:

```
Exemplu.metodaDeInstanta(); // Eroare
Exemplu obj = new Exemplu();
obj.metodaDeInstanta(); // Corect
```

Utilitatea membrilor de clasă

- **folosiţi pentru a pune la dispoziţie valori şi metode independente de starea obiectelor dintr-o anumită clasă.**

- **Declararea eficientă a constantelor**

```
class Exemplu {
    static final double PI = 3.14;
    // Variabila finala de clasa
}
```

- **Numărarea obiectelor unei clase**

```
class Exemplu {
    static long nrInstante = 0;
    Exemplu() {
        // Constructorul este apelat la fiecare instantiere
        nrInstante ++;
    }
}
```

- **Implementarea funcţiilor globale**

Agregare şi Compunere

- Se referă la prezenţa unei referinţe către un obiect dintr-o clasă într-o altă clasă.

- **Agregarea** (aggregation) – obiectul container poate exista şi în absenţa obiectelor agregate (care pot fi *null*) - *weak association*.

- **Compunerea** (composition) - este o agregare *strong* - *e*xistenţa unui obiect este dependentă de un alt obiect.
- La dispariţia obiectelor conţinute, existenţa obiectului container încetează.

Agregare şi Compunere

Iniţializarea obiectelor conţinute poate fi făcută în 3 momente de timp distincte:

- la **definirea** obiectului (înaintea constructorului: folosind fie o valoare iniţială, fie blocuri de iniţializare)
- în cadrul **constructorului**
- chiar **înainte de folosire** (*lazy initialization*)

Exemplu

```
class SetAsVector_A{
    private Vector v=new Vector();
        // sau prin constructor:
        /* private Vector v;
            public SetAsVector_A(){
                v=new Vector(); }*/
    public boolean add(Object o){
        if(v.contains(o)) return false;
        return v.add(o);
    }
    public String toString(){
        return v.toString();
    }
    /* trebuiesc redefinite toate metodele pe
care dorim sa le puna la dispozitie clasa!! */
  }
```

Exemplu

```
public static void main(String arg[]){
    SetAsVector_A s1= new SetAsVector_A();
    s1.add("nou");
    s1.insertElementAt("abc",0);
    /* trebuie definit  insertElement At în clasa
    SetAsVector !!!! */
    s1.insertElementAt("abc",0);
    System.out.println(s1);
    /* pentru că am definit  toString în clasa
    SetAsVector  */
     s1.v.remove(x); // nu este posibil, v private.
     s1.remove(x); // trebuie definit remove
}
```

Moştenire (Inheritance) - sinteza

- Numită şi **derivare**
- mecanism de refolosire a codului specific limbajelor orientate obiect
- reprezintă posibilitatea de a defini o clasă care **extinde** o altă clasă deja existentă.
- Ideea de bază este de a **prelua** funcţionalitatea existentă într-o clasă şi de a **adăuga** una nouă sau de a o **modela** pe cea existentă.
- Clasa existentă este numită **clasa-părinte**, **clasa de bază** sau **super-clasă**.
- Clasa care extinde clasa-părinte se numeşte **clasa-copil (child)**, **clasa derivată** sau **sub-clasă**.

Exemplu

```
class SetAsVector_M extends Vector{
   public boolean add(Object o){
        if(contains(o)) return false;
        return super.add(o);
   }
   /* trebuie redefinite toate metodele din clasa Vector care
   adauga sau modifica o valoare pentru a păstra condiţia de
   mulţime) nu permitem adăgarea de elemente duplicate!!!*/
} ….
public static void main(String arg[]){
   SetAsVector_M s1= new SetAsVector_M();
   s1.add(“aaa”);
   s1.insertElementAt(“abc”,0);
   s1.insertElementAt(“abc”,0); // corect?
   System.out.println(s1); }
```

Polimorfism (1)

- **Supraîncărcarea (overloading)**
- **Supradefinirea (overriding)**

```
class A {
    void metoda() {
        System.out.println("A: metoda fara parametru");
    }
    // Supraîncărcare
    void metoda(int arg) {
        System.out.println("A: metoda cu un
          parametru");
    }
}
class B extends A {
    // Supradefinire
    void metoda() {
        System.out.println("B: metoda fara parametru");
    }
}
```

Polimorfism (2)

O metodă supradefinită poate :

• să ignore codul metodei părinte:

```
B b = new B();
b.metoda();
// Afişează "B: metoda fara parametru"
```

• să extindă codul metodei părinte:

```
class B extends A {
// Supradefinire prin extensie
    void metoda() {
        super.metoda();
        System.out.println("B: metoda fara parametru");
    }
}
. . .
B b = new B();
b.metoda();
/* Afişează ambele mesaje:
"A: metoda fara parametru"
"B: metoda fara parametru" */
```

- **În Java nu este posibilă supraîncărcarea operatorilor.**

Controlul accesului şi supradefinirea

Următoarele reguli sunt obligatorii pentru metodele moştenite pe care dorim să le supradefinim:

- Metodele declarate **public** în superclasă trebuie să fie obligatoriu **public**e în toate subclasele.
- Metodele declarate **protected** în superclasă trebuie să fie **public**e fie **protected** în subclase. Ele nu pot fi declarate private sau implicit
- Metodele declarate **private** în superclasă nu se moştenesc, aşa că pentru ele nu există reguli de supradefinire!
- Metodele declarate fara modificator de acces (**implicit**) în superclasă trebuie să fie declarate **public**, **protected** sau **implicit** în subclase. Ele nu pot fi declarate private!

Regulă:

Putem supradefini cu un nivel de acces mai mare sau egal, dar nu mai mic (weaker)!